MVP in the Age of AI

Beyond ‘Vibe Coding’: Why writing code is cheap, but defining value is expensive.

Sravan Rekandar

Software is no longer the hard part

- **Generate code instantly** using AI.
- Build functioning apps in **days**.
- **Deploy faster** than ever before.

> ⚠ AI has removed execution friction. It has **NOT** removed **market risk**, user **misunderstanding**, or **trust issues**.

In 2025, code is cheap. Understanding the problem is expensive.

MVP is a thinking discipline, not a coding shortcut

Before AI

MVP was about reducing engineering costs (because coding was slow).

In the Age of AI

MVP is about reducing wasted time, false confidence, and lost credibility.

New Definition: The smallest product that delivers real value to a real user AND teaches you something important.

SMALLEST | REAL USER | LEARNING

A demo impresses classmates.
A product survives users.

❌

The Myth (Student Mindset)

- Full-stack app with 15 features
- We'll find users after we build
- Building before talking
- Confusing the demo with the product

✅

The Reality (Pro Mindset)

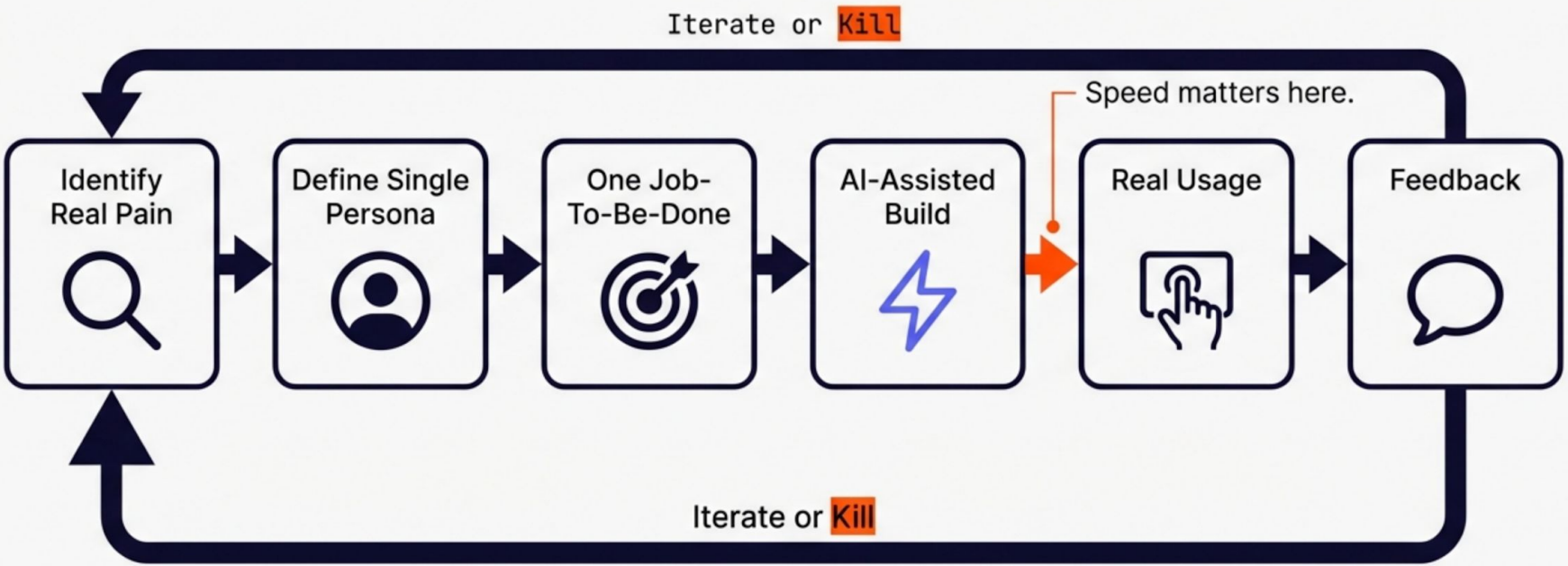
- Pain-first, not feature-first
- Usable, not just buildable
- Learnable (designed to generate feedback)

If users ignore it and you don't feel pain, it's not an MVP.

Vibe coding builds projects.
MVP thinking builds companies.

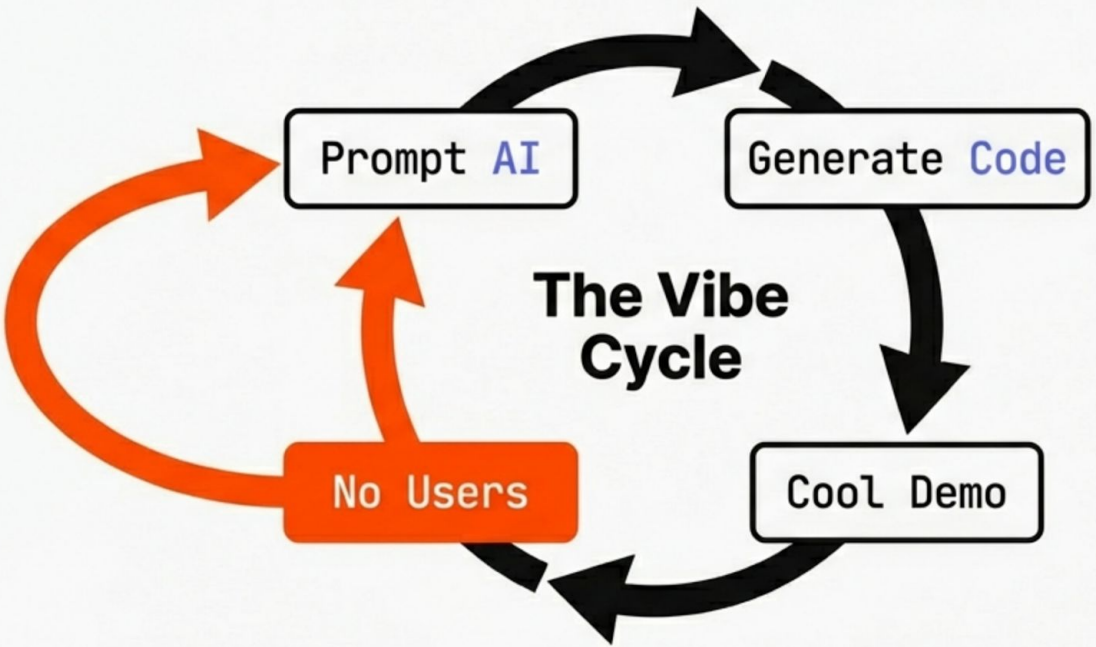
Vibe Coding	MVP Thinking
Looks cool	**Solves pain**
Many features	**One core value**
AI-first JetBrains Mono	**User-first**
Demo-driven	**Usage-driven**

The Core Flow: MVP is a loop, not a milestone.



The Trap: What is ‘Vibe Coding’?

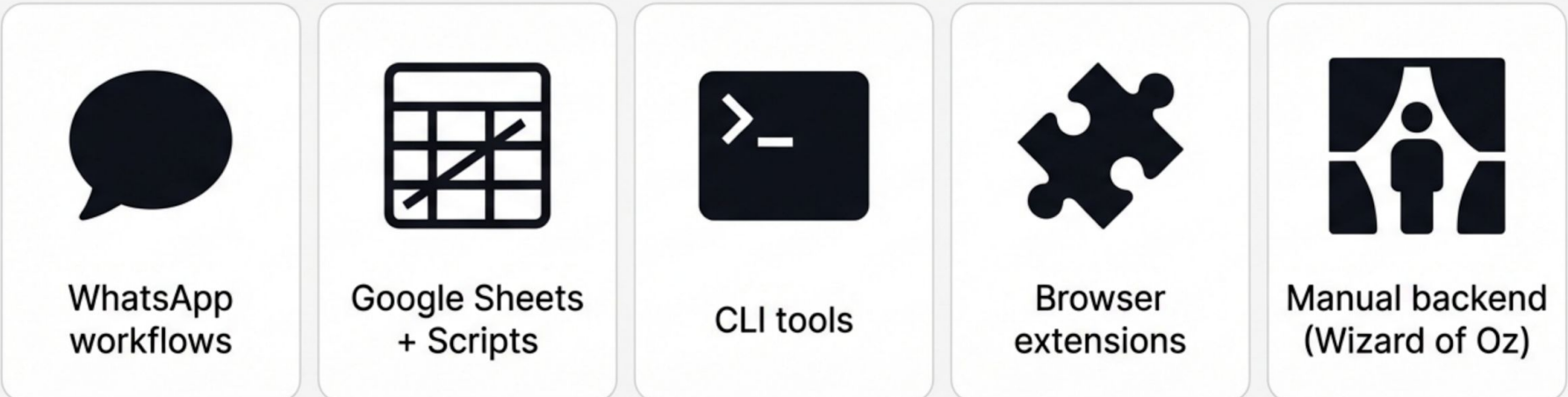
Building software by prompting AI, focusing primarily on speed and aesthetics, with minimal strategic thought given to the end user.



Symptoms

- Feature overload
- Zero feedback loop
- Optimised for the presentation, not the user

MVP does not mean full architecture



MVP ≠ Complete Architecture

Even AI giants start narrow

Airbnb

Solving Trust

Problem: People didn't trust listings.

The MVP: Founders took professional photos of apartments.

Tech: No platform rebuild. Just photos.

ChatGPT (OpenAI)

Solving Utility

The MVP: Text-only interface.

Tech: No plugins. No voice mode. No images.

Focus: Pure conversational utility.

Lesson: MVP removes friction, it doesn't add features.

AI accelerates the wrong direction too

AI is Excellent At

- Scaffolding code
- Rapid iteration
- UI generation
- Data transformation

AI is Terrible At

- Identifying real pain
- Understanding human context
- Saying "don't build this"

Strategic Advice: Use AI to speed up the build, never to skip the pain discovery.

The clash: Academic Mindset vs. Product Mindset

Syllabus completion	→	**Incomplete by design**
Final Submission	→	**Continuous feedback loop**
Marks-based success	→	**Usage-based success**

The Student Action Plan

- [ ] Talk to **10 real users**.
- [ ] Write problems in their **words**, not technical terms.
- [ ] Build only **one feature** that solves that problem.
- [ ] Ship in **7–14 days**.
- [ ] **Observe** usage, not opinions.

Screenshot this slide.

Killing an MVP is learning, not failure

When to Kill (The Signals)

- Users don’t return.
- Users don’t recommend it to others.
- You add features to hide the fact the core isn’t working.
- You avoid conversations with users.

If you aren’t willing to kill it, you aren’t testing it; you’re just defending it.

Anyone can code now.
Judgment is rare.

What CTOs Actually Look For

- ☐ 1. Can you define problems clearly?
- ☐ 2. Can you simplify aggressively?
- ☐ 3. Can you ship and listen?
- ☒ 4. Can you **STOP** **building**?

The real MVP is not your product. It’s your clarity of thinking.

Stop vibe coding. Start solving.

Y Combinator Library

Paul Graham Essays

Marty Cagan (Product Thinking)

Airbnb Growth Story